

Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales



Arquitectura de computadoras
Practico N°1: ALU

Autores
Delamer Ignacio
Rivarola A. Ignacio

Introducción

En este trabajo práctico implementamos un sistema digital capaz de comunicarse en serie mediante el protocolo UART, integrando la Unidad Aritmético-Lógica (ALU) desarrollada en el TP1.

El objetivo principal es realizar una comunicación exitosa. Esto quiere decir que se debe:

1. Recibir por UART, desde un dispositivo externo, los operandos y el código de operación.
2. Procesar esa información con la ALU ya diseñada.
3. Devolver el resultado y las banderas de estado nuevamente a través de la interfaz serie.

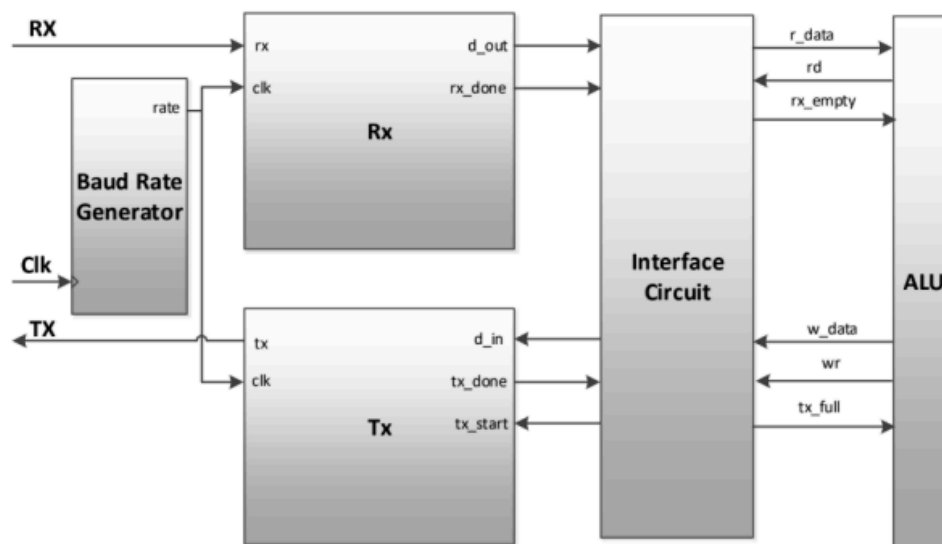
Para lograrlo construimos una arquitectura modular, basada en máquinas de estado finitas (FSM) y sincronizada por un generador de baud rate común. Todo el sistema utiliza reset síncrono para asegurar una inicialización ordenada y reproducible.

El desarrollo nos permitió reforzar conceptos clave de la materia tales como el diseño secuencial, comunicación asíncrona, control mediante FSM y la integración de bloques previamente diseñados en una solución completa orientada a hardware programable.

Desarrollo

Visión general del sistema

La implementación se organizó en bloques claramente diferenciados, conectados entre sí mediante señales de datos y control. A continuación se presenta un diagrama de bloques en el cual observamos la planificación general del proyecto.



Componentes principales:

- **Baud Rate Generator:** genera el pulso de referencia utilizado por los módulos UART para temporizar la transmisión y la recepción.
- **UART Receiver (Rx):** toma el flujo serie de la línea RX y lo reconstruye en forma de byte, indicando cuándo el dato recibido es válido.
- **UART Transmitter (Tx):** envía en formato serie un byte entregado en paralelo cuando se le ordena iniciar la transmisión.
- **Módulo de interfaz / control:** dirige la secuencia de recepción de operandos y OpCode, habilita el cálculo en la ALU y coordina el envío del resultado y flags.
- **ALU:** ejecuta las operaciones aritmético-lógicas definidas en el TP1.
- **Módulo top (uart_alu_top):** integra todos los bloques y define el comportamiento global del sistema.

Esta descomposición modular permitió verificar cada bloque de forma aislada previo a la integración, reduciendo errores y simplificando ajustes posteriores

Descripción de Módulos

1. Baud_gen

El módulo baud_gen forma parte del clock principal de la placa y produce un “tick” periódico que se utiliza como referencia temporal para el emisor y el receptor UART.

El período del tick se calcula en función de la frecuencia de reloj (50 Mhz) y del baud rate objetivo, utilizando un factor de sobremuestreo $\times 16$ para mejorar la robustez del muestreo de bits. Este valor se calcula de la siguiente forma: $f_{clk} / (\text{baud} \times 16)$.

Cada vez que el contador interno alcanza el valor máximo configurado, se reinicia y genera un pulso de un ciclo, empleado por uart_rx y uart_tx para avanzar sus FSM de recepción y transmisión.

2. Uart_rx Receptor

El receptor se implementó como una FSM que sigue la estructura típica del protocolo:

- **IDLE:** espera que la línea rx caiga a nivel bajo, indicando un posible bit de inicio.
- **START:** verifica que el nivel bajo se mantenga en el instante correcto de muestreo.
- **DATA:** lee secuencialmente los bits de datos, comenzando por el LSB.
- **STOP:** comprueba el bit de parada en nivel alto.
- **DONE:** almacena el byte reconstruido y activa la señal rx_done durante un ciclo.

Si las condiciones del bit de inicio o de stop no se cumplen, el módulo vuelve a IDLE sin validar el dato. Esto garantiza que solo se acepten tramas bien formadas.

3. Uart_tx Transmisor

El transmisor también se resolvió con una FSM con estados:

- **IDLE:** espera la orden de transmisión (tx_start).
- **START:** envía el bit de inicio (0).
- **DATA:** transmite los 8 bits del byte a enviar, LSB primero.
- **STOP:** genera el bit de parada (1).

- **DONE:** indica fin de envío mediante tx_done.

Internamente utiliza contadores para mantener cada bit activo durante el tiempo correspondiente al baud rate definido. El dato de entrada se registra al inicio de la transmisión para evitar cambios durante el envío.

4. ALU

La ALU recibe dos operandos de 8 bits y un código de operación. Entre las operaciones soportadas se encuentran las mismas que en el TP1, es decir:

- ADD, SUB
- AND, OR, XOR, NOR
- SRL, SRA

En función del OpCode, la ALU produce el resultado y actualiza las banderas de estado (por ejemplo, carry y zero), que luego serán enviados por UART.

Todo este diseño, se realizó implementando el módulo ALU diseñado en el trabajo práctico anterior.

5. Interface

La interfaz actúa como “cerebro” entre UART y ALU.

Recibe los tres bytes correspondientes a A, B y el código de operación (OpCode). Su FSM controla:

- **Recepción de A**
- **Recepción de B**
- **Recepción del OpCode**
- **Disparo del cálculo en la ALU**
- **Envío del resultado**
- **Envío de las flags**

Entre cada etapa se sincroniza con rx_done y tx_done para asegurarse de que los datos fueron recibidos o transmitidos correctamente antes de avanzar. De esta forma se evita superposición de tramas o pérdida de información.

Los estados S_GET_A, S_GET_B y S_GET_OP controlan la recepción secuencial de los operandos y el código de operación desde el PC. Luego, la FSM pasa a S_SEND_RESULT, donde se envía el resultado calculado por la ALU, y finalmente a S_SEND_FLAGS, donde se transmiten las banderas carry y zero. Entre cada envío o recepción, los estados S_WAIT_RESULT y S_WAIT_FLAGS garantizan la correcta sincronización con las señales rx_done y tx_done.

6. Uart_alu_top (Modulo TOP)

Este módulo se encarga de agrupar el resto de módulos, permitiendo así su trabajo en conjunto.

Una vez realizada esta agrupación, podemos definir el flujo general de trabajo:

1. Espera los bytes de entrada (A, B y OpCode).
2. Habilita la operación en la ALU.
3. Ordena el envío del resultado y, a continuación, de las banderas.

Mediante una FSM interna, gestiona los estados:

- WAIT_A: espera el primer operando.
- WAIT_B: espera el segundo operando.
- WAIT_OP: espera el código de operación.
- SEND_R: envía el resultado.

Este módulo es el punto de integración física con la FPGA y con el entorno externo. Se encarga de permitir la comunicación con un PC, el cual ejecuta un script de python a fin de facilitar las pruebas

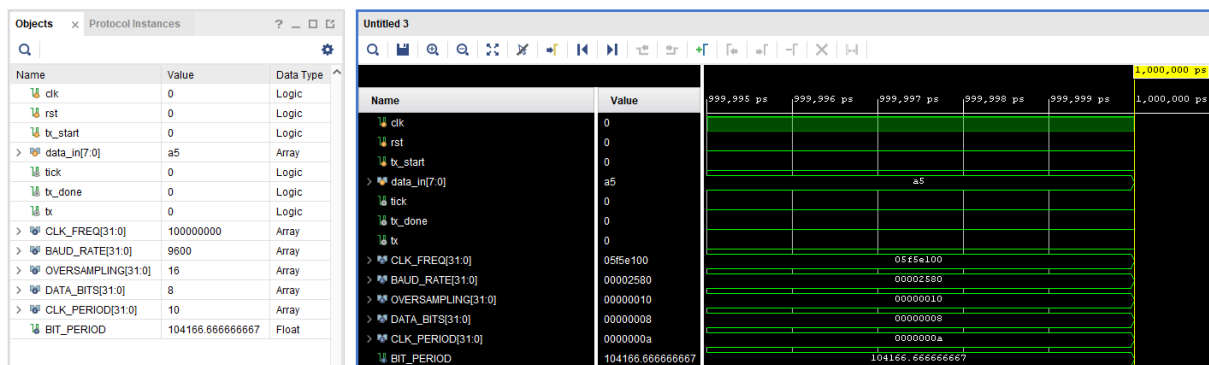
Verificación mediante Testbench

La validación se realizó principalmente por simulación, usando testbenches específicos para cada bloque y para el sistema completo.

tb_uart_tx

Se comprobó:

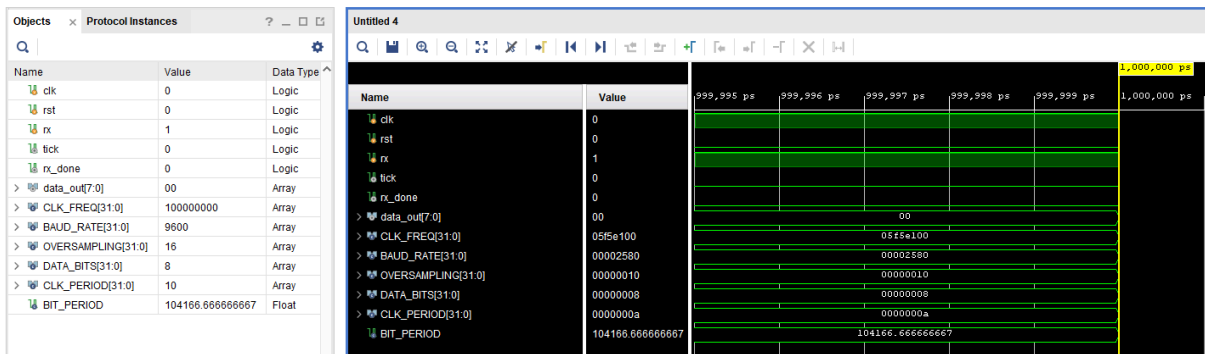
- Que la secuencia en la línea tx respete: start bit → 8 bits de datos → stop bit.
- Que la duración de cada bit coincida con el baud rate.
- Que tx_done se active sólo al finalizar el envío del byte completo.



tb_uart_rx

En este banco de pruebas se generó una trama serie conocida en la línea rx, sincronizada con los ticks del baud_gen, y se comprobó que:

- El byte reconstruido en data_out coincida con el enviado.
- rx_done se active exactamente al completar la recepción.
- La FSM recorra correctamente sus estados para start, data y stop.

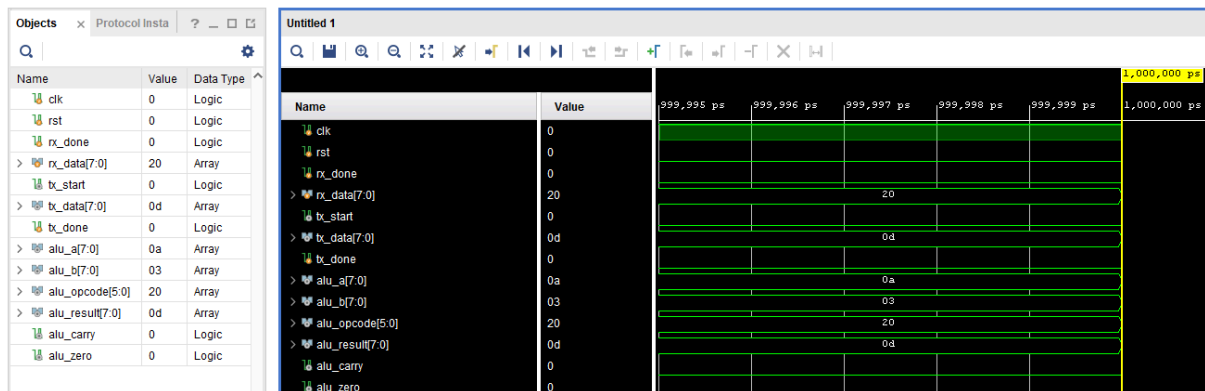


tb_interface

Aquí se realizó el testeo de el flujo completo controlado por la interfaz:

- Recepción ordenada de A, B y OpCode.
- Habilitación de la ALU.
- Envío secuencial del resultado y las flags por el transmisor UART.

Se analizaron los cambios de estado, las señales de control hacia ALU y las señales tx_start / rx_done para corroborar la coordinación entre bloques.



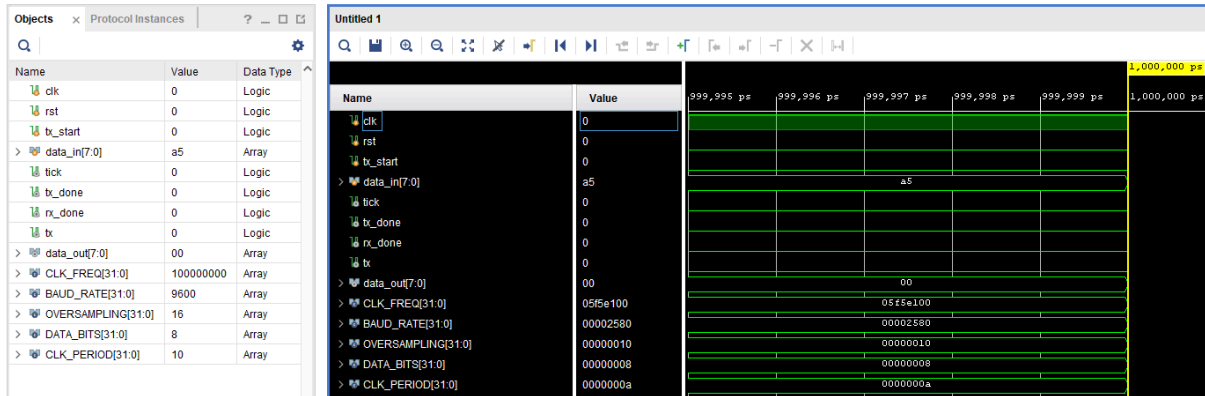
tb_uart_loopback

En modo loopback se conectó tx con rx internamente:

- Se envió un byte por uart_tx.
- Se verificó que uart_rx recibiera el mismo valor.

- Se confirmó la consistencia del baud rate y del sobremuestreo.

Este testeo permitió comprobar la transmisión y recepción completas sin depender de hardware externo adicional.

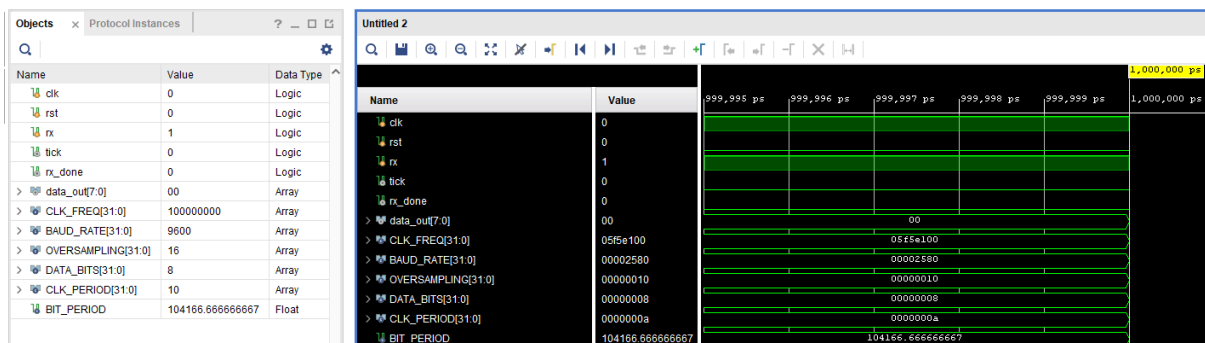


tb_uart_alu_top

Finalmente se simuló el sistema integrado:

1. Envío de A, B y OpCode por UART.
2. Procesamiento en la ALU.
3. Retorno del resultado y las flags por el mismo enlace.

Las formas de onda mostraron la correcta interacción entre las FSM, la actualización de registros de datos y la coherencia entre operación solicitada y resultado obtenido.



Herramienta de verificación por PC (script Python)

Para verificar el diseño sobre la FPGA se desarrolló un script en Python que se comunica por el puerto serie con la placa.

El programa permite:

- Seleccionar una operación de la ALU.
- Ingresar operandos A y B desde la consola.
- Enviar los tres bytes (A, B, OpCode) hacia la FPGA.
- Recibir y mostrar el resultado y las flags.

Esto habilita una prueba “en vivo” del sistema, más cercana al uso real del dispositivo.

El script permite realizar las siguientes funciones:

- Mostrar una estructura/tabla que asocia cada operación con su nombre y OpCode.
- Mostrar un menú para que el usuario elija la operación.
- Funciones para leer operandos en decimal o hexadecimal, validando el rango.
- Una rutina que envía los tres bytes por UART y espera la respuesta completa (resultado + flags), con manejo de timeout y limpieza de buffers.
- Una función de presentación que muestra la información en decimal, hexadecimal y binario, interpretando adecuadamente las flags. En el caso de restas, contempla representación en complemento a dos cuando corresponde.

El bucle principal mantiene la conexión con la FPGA y permite realizar múltiples operaciones consecutivas.

A la hora de utilizarlo junto al hardware, su workflow es el siguiente:

PC (Python) → UART → FPGA (uart_rx → interface → ALU → uart_tx) → UART → PC.

Este esquema permitió validar simultáneamente:

- La correcta configuración del baud rate.
- La integridad de los datos transmitidos.
- El comportamiento funcional de la ALU integrada en la FPGA

Resultados y correcciones

En simulación:

- Se observó el comportamiento esperado de todos los módulos.
- Las tramas UART se generaron y recibieron correctamente.
- El sistema UART + ALU respondió con los resultados correctos para los distintos OpCodes.

En la implementación sobre placa se observó un problema, en caso de que se deseara implementar la uart de esta manera:

- Los datos (luego de ser procesados por la alu) no se guardaban en un registro, sino, que eran enviados directamente por el modulo uart hacia el pc

Conclusión

Como grupo logramos implementar y validar un sistema UART–ALU completamente funcional sobre FPGA, partiendo de la ALU del TP1 e incorporando una interfaz serie robusta y controlada mediante FSM.

Los puntos más importantes que nos llevamos son:

- La utilidad del diseño modular para escalar complejidad sin perder claridad.
- La relevancia de sincronizar cuidadosamente las señales entre bloques secuenciales y combinacionales.
- La importancia de complementar la simulación con pruebas reales en placa para detectar problemas de sincronismo que no siempre son evidentes en el entorno idealizado.

Este trabajo nos permitió consolidar criterios de diseño digital profesional y comprender mejor cómo integrar comunicación serie, lógica combinacional y lógica secuencial en un único sistema coherente y funcional.